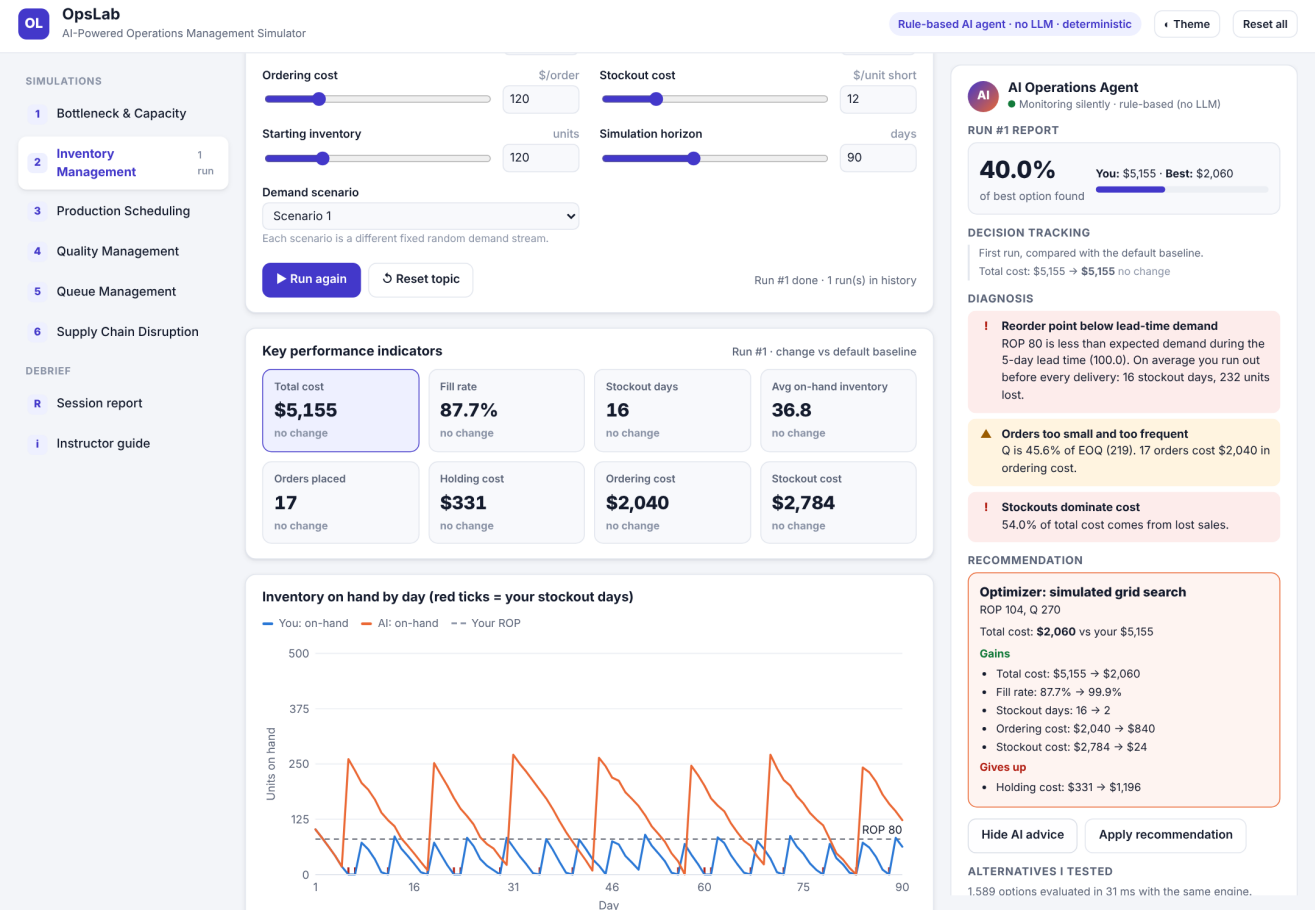
OpsLab

AI-Powered Operations Management Simulator

Project documentation · MBA Operations Management / Operations Analytics



Inventory Management topic after a run, with the AI Operations Agent’s advice open (right).

Deliverable	File
Interactive simulator (single self-contained page)	opslab.html
This document	docs/OpsLab_Documentation.pdf
Quick start and instructor notes	README.md

1. Problem

Operations concepts such as bottlenecks, safety stock, dispatching rules, cost of quality, queueing and supply-chain resilience are usually taught with static formulas and spreadsheet exercises. Students rarely get to **make a decision, see the consequences, and compare their choice with a better one** in the same session. Commercial simulation games are expensive, need accounts, and are hard to run live in a 75-minute class.

2. Solution

OpsLab is a small virtual factory in one HTML file. The student acts as an operations manager: they set the decisions for each topic, press **Run simulation**, and get KPIs, a chart, a before/after comparison and a decision history. The **OpsLab Agent** monitors every run without interrupting. After each run it diagnoses inefficiencies, tests feasible alternatives with the same simulation engine, recommends the best one and explains the trade-offs. The **Session report** summarizes decisions, mistakes, improvements and lessons learned.

Design principles

- **Every number is computed.** All KPIs come from deterministic formulas or a seeded simulation. Nothing is hard-coded or made up.
- **Decisions vs scenario.** Inputs are split into decisions (blue, what the manager controls) and scenario parameters (the environment). The agent optimizes only decisions.
- **Fair comparisons.** Alternatives are evaluated on the same scenario and the same random demand stream (common random numbers).
- **No interruptions.** The agent reports after each run. Detailed advice appears only on request (Get AI advice).
- **Zero setup.** No login, backend, internet connection or paid API. Open the file in any modern browser.

3. Topics, models and formulas

Each topic has adjustable inputs with sensible defaults, a Run / Run again button, a Reset button, 8 KPI cards, a chart (yours vs the AI recommendation), a before/after table and a decision history.

Case data. Every topic opens with a written case brief and numbered exhibits holding all the data needed to solve it, downloadable as CSV. Inventory gives 90 days of demand history from the same process (and, after the run, the actual demand for the debrief). Queues give a 40-customer observation log. Process time gives a 10-cycle time-study sheet with rating and allowance. Quality includes a prevention-effectiveness table. The exhibits follow the current scenario, generated cases and campaign rounds, and keep hidden outcomes hidden.

3.1 Bottleneck & Capacity

Three stations (Cutting 4 min, Assembly 7 min, Packing 5 min per unit), demand 100 units per 480-min shift. Decision: parallel machines per station (1–4). Objective: maximize shift profit.

Quantity	Formula
Capacity	`Cap_i = machines_i × shift ÷ time_i`
Throughput	`TH = min(min Cap_i, Demand)`
Utilization	`U_i = TH ÷ Cap_i`
Cycle time	`CT = max(time_i ÷ machines_i)`
Line efficiency	`Σ time_i ÷ (Σ machines_i × CT)`
Profit	`TH × margin − Σ machines × machine cost`

Default vs agent: Default 1/1/1 earns $1,340 (Assembly is the bottleneck at 68 units). The agent finds 1/2/1: $1,920. Adding a Packing machine as well meets all demand but earns less ($1,900).

3.2 Inventory Management

Continuous-review (ROP, Q) policy, simulated day by day for 90 days with normally distributed demand (mean 20, s.d. 6) from a seeded generator, 5-day lead time and lost sales. Decisions: reorder point and order quantity. Objective: minimize holding + ordering + stockout cost.

Quantity	Formula
EOQ	$\sqrt{(2DS \div H)}$
Safety stock	$z \times \sigma_d \times \sqrt{L}$
Reorder point	$\bar{d} \times L + SS$
Total cost	$\Sigma\ H \times \text{on-hand} + \text{orders} \times S + \text{short units} \times p$
Fill rate	$\text{units sold} \div \text{units demanded}$

Default vs agent: Default ROP 80 / Q 100 costs $5,155 (fill rate 87.7%). The textbook policy (EOQ 219, ROP 122) costs $2,209. The agent's grid search over 1,500+ policies finds ROP 104 / Q 270: $2,060.

3.3 Production Scheduling

Six jobs on a single machine with editable processing times and due dates. Decision: rule (FCFS, SPT, EDD, CR, Slack, LPT) or a custom sequence. Objective: minimize late penalty ($/job-day), with average flow time as the tie-breaker.

Quantity	Formula
Completion	$C_j = \Sigma$ processing times up to job j
Tardiness	$T_j = \max(0, C_j - d_j)$
Avg jobs in system	$\Sigma\ F_j \div \text{makespan}$
Penalty	$\Sigma\ T_j \times \text{penalty}$

Default vs agent: FCFS: 4 late jobs, $22,000. EDD: $4,500. The agent enumerates all 720 sequences and finds F-B-D-A-E-C: $4,000. It also tests Moore–Hodgson (fewest late jobs).

3.4 Quality Management

A 5,000-unit batch with a 6% base defect rate. Decisions: inspection coverage (%) and prevention spend. Prevention reduces defects exponentially toward a process floor. Objective: minimize total cost of quality (COQ).

Quantity	Formula
Defect rate	$p = p_{min} + (p_0 - p_{min}) \cdot e^{(-Prev/1000)}$
Caught	$N \cdot p \times \text{coverage} \times \text{detection}$
Escaped	$N \cdot p - \text{caught}$
COQ	$\text{prevention} + \text{appraisal} + \text{rework} + \text{scrap} + \text{external failure}$
Sigma level	$z(1 - DPMO/10^6) + 1.5$

Default vs agent: Default (100% inspection, no prevention) has a COQ of $10,220. The agent finds $3,500 of prevention with no inspection: $7,497. This illustrates "you cannot inspect quality in".

3.5 Queue Management

M/M/c service desk: 50 arrivals/hour, 4-minute mean service. Decision: number of servers. Objective: minimize wages + waiting cost, subject to a service-level target (80% of customers wait less than 2 min).

Quantity	Formula
Utilization	$\rho = \lambda \div (c\mu)$
Erlang C	$P_w = [a^c/c!\cdot 1/(1-\rho)] \div [\Sigma\, a^k/k! + a^c/c!\cdot 1/(1-\rho)]$
Queue	$L_q = P_w\cdot\rho/(1-\rho),\ W_q = L_q/\lambda$
Service level	$1 - P_w\cdot e^{-(c\mu-\lambda)T}$

Default vs agent: 4 servers: $252/h and only 53% served within 2 min. 5 servers: $176/h and 86%, which meets the target. Unstable choices ($\rho \geq 1$) are flagged as infinite queues.

3.6 Supply Chain Disruption

60-day daily simulation with a base-stock policy. The primary supplier shuts down on days 10–24. Decisions: backup supplier share (dual sourcing), buffer stock (days), contingency plan on/off and response time. Objective: minimize backup premium + holding + lost margin.

Quantity	Formula
Order-up-to	$S = d \times L_{eff} + SS$
Daily order	$\max(0,\ S - \text{inventory position})$
Stuck orders	arrive at end of outage + L_1
Emergency cap	$\beta\cdot d$ + spare backup capacity
Cost	premium × backup units + holding + lost units × margin

Default vs agent: Default (single source, 3-day buffer, no plan) costs $20,976 with 480 units lost. The agent tests 1,716 combinations and finds 10% dual sourcing, a 5-day buffer and a 1-day contingency switch: $5,524, 100% fill rate.

3.7 Process Time Analytics

Order-to-ship flow: order entry → cutting → transport → assembly → inspection → packing, 60 units/day in 480 min. Assembly time comes from a time study (observed 5.0 min × 110% rating × 1.15 allowance = 6.32 min). Decisions: batch size, SMED (setups −60%), cellular layout (transport −80%), poka-yoke (removes final inspection). Objective: minimize daily cost of lead time + unmet demand + improvements.

Quantity	Formula
Standard time	observed × rating × (1 + allowance)
Takt time	available time ÷ demand
Utilization	$u = D \times (t + S/Q) \div T$
Queue (Kingman)	$((ca^2+ce^2)/2) \times u/(1-u) \times (S + Q\cdot t)$
Lead time	order delay + $(Q-1)/2\cdot$takt + Σ(queue + S + Q·t) + transport
PCE / Little	$PCE = VA \div LT,\ WIP = TH \times LT$

Default vs agent: Default batch 40 gives a 43.4-hour lead time, PCE 0.4%, 325 units of WIP and $2,601/day. Small batches overload assembly unless setups are cut. The agent tests 800 combinations and finds batch 9 with SMED, cell and poka-yoke: 14.6 hours, 110 units of WIP, $1,068/day.

3.8 Time Series Analysis (ARIMA): stepwise lab

Topic 8 follows the ARIMA sessions (Session 03–04): a guided Box–Jenkins exercise in eight steps, each requiring a decision from the player. (1) Visualise the 365-day series and judge stationarity; (2) run the ADF test and conclude at 5%; (3) choose a transformation (none, first or second difference,

log); (4) re-test and confirm or difference again; (5) read the ACF; (6) read the PACF; (7) specify and fit up to three ARIMA(p,d,q) models on 335 training days and forecast the 30 test days; (8) choose the model to forecast with. The Analytics Agent runs only the analyses requested, checks each decision against the evidence (accepting every reading a sample plot supports), and waits for the next input. It never advances or gives the answer. Wrong choices are allowed and show up in later steps; a slowly decaying ACF triggers a non-stationarity warning with a way back to Step 3. The debrief reveals the generating process, an automatic AIC grid search (p, q ≤ 3), a scorecard out of 8 and a 14-day forecast.

Dataset	Generating process
Oxygen cylinders per day, hospital	AR(2), stationary
Milk demand (kg), dairy cooperative	ARIMA(0,1,1) with drift
Parcels per day, sorting hub	ARIMA(1,1,0) with drift
Service tickets per day, call centre	MA(2), stationary
Vegetable demand (kg), grocery hub	ARIMA(1,1,1) with drift

Statistics run in the browser: ADF with AIC lag selection and MacKinnon p-values and critical values; ACF with Bartlett bands; PACF by Durbin–Levinson; ARIMA by conditional least squares with stationarity and invertibility enforced. Checked against Python statsmodels on all five datasets: ADF statistic, p-value, lags, observations, critical values, ACF and PACF match exactly, and ARIMA coefficients agree with statsmodels' maximum-likelihood estimates to about ±0.03.

3.9 Study Room: learn from session slides

Students upload a session's PowerPoint (.pptx; several at once; .txt/.md also accepted). The file is parsed in the browser: the .pptx is unzipped with the built-in DecompressionStream, and slide titles, text, tables, images and speaker notes are extracted in presentation order, skipping footers, dates and slide numbers. Tested on the course decks (Sessions 01–11): slide counts match python-pptx exactly, about 300 images were extracted, and all five decks parsed in under 2 seconds.

Tab	OpsLab Agent (built in)	Agent + AI model (any provider, incl. Ollama)
Notes	Sections, formulas, tables, code blocks, key-term glossary with slide references; download .md	Explained study notes with slide citations and key takeaways, written in parts for long decks
Visual	Slide viewer with the slide's images; diagram of the slide structure; links to the matching OpsLab simulation	Concept map (validated JSON), plain-words explanation, analogy and example
Quiz	Fill-in-the-key-term questions with explanations; score, best score, retry missed	Multiple-choice questions (concept, interpretation, calculation), validated, with slide references
Help	Most relevant slides and sentences (TF-IDF retrieval)	Answer grounded in the retrieved slides, with citations

Slide text is sent only when an AI button is pressed, and only to the chosen provider; with Ollama it stays on the student's computer. Old .ppt and PDF files are not supported yet.

3.10 Campaign mode: quarters or years

Every topic can be played as a multi-round campaign: quarterly (4 or 8 rounds) or yearly (3 or 5 rounds). Each round's briefing changes the scenario, e.g. Q4 demand peaks and growth, inflation, a supplier lead-time shock, a new product launch with a learning curve, a product recall, wage agreements and service targets, a new operator re-rated by time study, or a supplier shutdown that is revealed only after the plan is committed. Scenario inputs are locked to the round; decisions carry over and each round is played once. Round results are scaled to the period (e.g. profit per shift × 63 shifts per quarter) and summed. The total is compared with the AI's best plan for each round, which is labelled as a hindsight benchmark. The campaign view shows round chips, the briefing, cumulative totals, a round-by-round chart and table, and a final summary naming the best and hardest rounds. Campaign results appear in the session report, and the class-room server accepts official campaign rounds as fair results.

Topic	Value per round (quarter / year)
Bottleneck	profit per shift × 63 / 250
Inventory	cost per 90-day run × 1 / 4
Scheduling	penalty per weekly schedule × 13 / 52
Quality	cost of quality per weekly batch × 13 / 52
Queues	cost per hour × 504 / 2,000
Supply chain	cost per 60-day window × 1.5 / 6
Process time	cost per day × 63 / 250

3.11 Scenario Agent

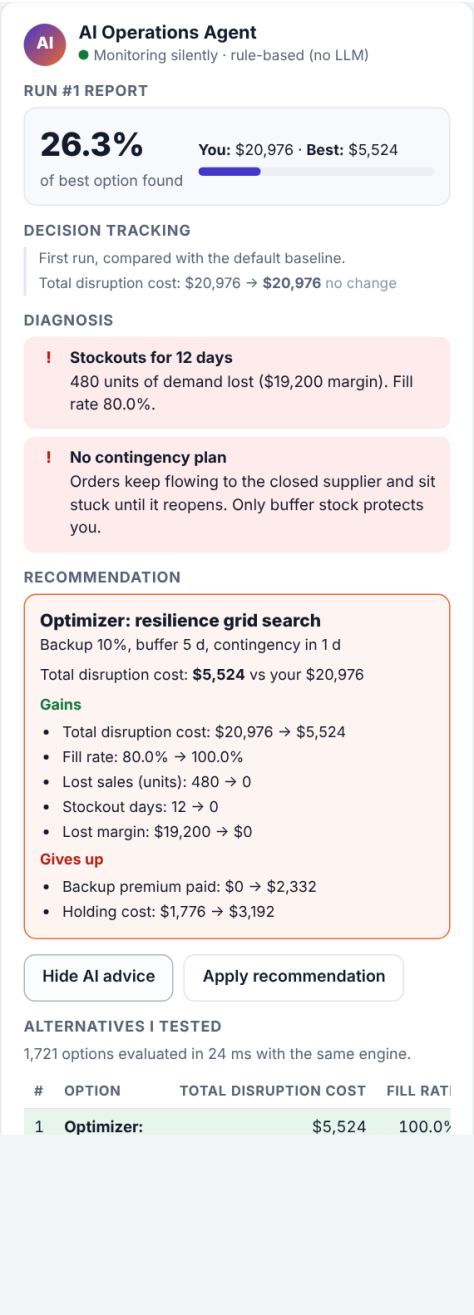
One Scenario Agent serves every topic. Given an optional brief (e.g. “monsoon floods hit a pharma plant in Vizag”), it asks the connected AI model for a case in JSON: a title, a short story, and new values for the topic’s scenario inputs. It is told the input ids, units, ranges and defaults, the topic’s concept, and that decisions are off limits. OpsLab requests JSON mode where the provider supports it (Ollama, OpenAI, Groq, Gemini), then repairs the slips small models make (missing or trailing commas, unescaped quotes, comments, line breaks in text, output cut off mid-way). If the reply is still unusable, it sends the model its own reply and the exact error and asks for a corrected version, up to two times. It then drops unknown and decision inputs and fits every number to its range and step. For campaigns it writes a briefing and conditions for every round; for the supply-chain topic the disruption stays hidden until the round is played. Without a model connected, its built-in generator creates seeded random variations. Generated cases are practice in the class room; official scenarios decide the leaderboard.

Ollama option: one click selects the Ollama provider, lists the installed models (skipping embedding models), and picks a suitable chat model. When OpsLab is opened from the class server, requests go through the server, so a single Ollama on the professor’s laptop serves the whole class for free.

4. AI Operations Agent workflow

The **OpsLab Agent** runs in the browser and is deterministic, so results are reproducible and never made up. After each run:

Step	What the agent does
1. Track	Logs the run, the decisions changed since the previous run (scenario changes marked ⚑) and the movement of the main KPI.
2. Diagnose	Applies operations rules, e.g. ROP below lead-time demand, utilization above 85%, external failure above 40% of COQ, a long job blocking an urgent one, no contingency plan. Some rules test the next-smaller option first, so the agent never flags a choice that is already optimal.
3. Generate	Builds feasible alternatives: textbook heuristics (EOQ + 95% ROP, SPT/EDD/CR, Moore–Hodgson, staff-to-demand) plus an exhaustive or grid search over the decision space (30 to 1,700 options per topic).
4. Evaluate	Runs every alternative through the same simulation engine and scenario.
5. Compare	Ranks on the topic objective (profit, cost or penalty), respecting constraints such as the service-level target. Lists what the best option gains and what it gives up (cost, output, delays, service).
6. Recommend	Shows the share of the best option achieved, the recommendation, a trade-off explanation and topic lessons. “Apply recommendation” copies its decisions into the inputs; the student still presses Run.
7. Report	The Session report compiles, per topic: decisions made, mistakes found (with run numbers), improvements and setbacks, share of best achieved, and lessons learned. It can be printed or saved as PDF.



Agent panel

Right-hand panel after a Supply Chain run: score versus the best option found, decision tracking, diagnosis, and the recommendation with gains and give-ups. Get AI advice adds the ranked table of alternatives tested, the trade-off explanation and lessons.

This panel needs no API key. The agent chat below the panel builds on the same engine and optimizer.



Quality topic: cost-of-quality breakdown, your plan vs the AI recommendation.

Optional live AI agent (LLM chat)

The **Ask the AI agent** chat box lets the agent work through an AI model from many providers. It answers free-form questions such as “Should I add a machine at B?” or “What if demand rises 30%?” with a tool-calling loop of up to 8 steps. No backend is needed.

Provider	API format	Notes
Anthropic Claude	Claude Messages API	Default claude-opus-5-5; direct from browser
NVIDIA NIM (build.nvidia.com)	OpenAI-compatible	e.g. nvidia/nemotron-3.5-super-vl-120b-a12b; needs the local proxy (CORS)
OpenAI, Groq, OpenRouter, Google Gemini	OpenAI-compatible	Usually direct; use the proxy if the browser is blocked
Ollama (local)	OpenAI-compatible	Free, no key; start with OLLAMA_ORIGINS="*" ollama serve
Any other endpoint	OpenAI-compatible	vLLM, self-hosted NIM, Together, Mistral…: set the base URL

Tool	Purpose
get_current_state	Inputs with ranges and roles, the latest run’s KPIs and diagnosis, and the agent’s best option.
run_simulation	Sandbox what-if on the deterministic engine. It never changes the student’s inputs or history.
optimize	Runs the agent’s optimizer, optionally under changed scenario inputs, to check robustness.
propose_settings	Shows an “Apply suggested settings” button. The student decides whether to apply and run.

Guardrails: the system prompt requires that every number comes from a tool result; invalid inputs come back as tool errors; scenario-only and decision-only checks apply; it handles errors (bad key, wrong model, rate limit, blocked browser call). Models without tool calling are detected automatically and switched to grounded mode, where OpsLab sends the pre-computed simulator data with the question. Keys stay in the browser, or pass through the user’s own local proxy (tools/llm-proxy.mjs: localhost only, allowlisted hosts). Without a model connected, the OpsLab Agent answers from its own analysis, so the basic demo needs no paid API.

Optional class room

One command on the professor’s laptop, **node tools/opslab-server.mjs** (Node 18+, no dependencies), turns OpsLab into a class activity on the local network. The professor creates a room (optionally restricted to a pasted roll-number list) and shares a 6-character code. Students open the printed address, join with the code and their roll number, and every run is submitted automatically.

Feature	How it works
Live dashboard	Best % of optimum per student and topic, leaderboard (score out of 7), class first-run vs best-run averages, most common mistakes per topic, students not yet joined; refreshes every 10 s.
Fair results	The server loads the simulation engine from opslab.html and re-computes every run from the submitted decisions; out-of-range inputs are rejected. Only standard-scenario runs count for the leaderboard.
Identity	One device per roll number (token-based); the professor can reset a device. Roll numbers outside the class list are refused.
Robustness	Runs made while the server is unreachable are queued and sent later; data is saved atomically to opslab-rooms.json with a .bak copy and flushed when the server stops (Ctrl+C is safe); rooms can be closed and reopened.
Grading	CSV export with rank, roll, name, score, best % and runs per topic, including students who never joined.

5. Technologies

- One HTML5 file (about 2,100 lines) with embedded CSS and vanilla JavaScript. No frameworks, libraries, CDNs or network calls.
- Charts are hand-built inline SVG (grouped bars, lines, Gantt, stacked bars) with hover tooltips.
- Seeded PRNG (mulberry32) + Box–Muller for reproducible demand; Acklam inverse-normal for sigma levels; exact Erlang C.
- Responsive CSS grid (desktop three-column, tablet two-column, phone single column) with light and dark themes and print styles for the report.
- Optional live agent: two adapters, Claude Messages API (default Claude Opus 5.5) and OpenAI-compatible chat completions (NVIDIA NIM, OpenAI, Groq, OpenRouter, Gemini, Ollama, custom), sharing one client-side tool-calling loop over four simulator tools. An optional zero-dependency Node proxy (tools/llm-proxy.mjs) handles providers that block browser calls.
- Optional class-room server: Node 18+ built-ins only (http, vm, crypto); serves the page, hosts rooms, re-computes runs with the engine extracted from opslab.html, and includes the LLM proxy.
- Validation: automated headless-browser tests (Playwright + Chromium).

6. Usage instructions

- Open **opslab.html** in Chrome, Edge, Firefox or Safari. It also works offline.
- Pick a topic in the left navigation. Open Concept & formulas to review the theory.
- Change the blue **decision** inputs (sliders or number boxes). Grey **scenario** inputs change the environment.
- Press **Run simulation**. Read the KPI cards (▲/▼ vs the previous run), the chart and the before/after table.
- Read the agent's report on the right. Press **Get AI advice** for the alternatives table, or **Apply recommendation** and then **Run again**.
- **Reset topic** restores defaults and clears that topic's history. **Reset all** clears the whole session.
- Optional: open **AI model settings** in the Ask the AI agent box, choose a provider (Claude, NVIDIA, OpenAI, Groq, OpenRouter, Gemini, Ollama, or custom), enter the key, click **Load models** to pick a current model, and ask questions in plain language. For NVIDIA, first run **node tools/llm-proxy.mjs** and keep "Use local proxy" ticked. Without a model connected, the OpsLab Agent answers.
- Optional class room: the professor runs **node tools/opslab-server.mjs**, opens the printed address, creates a room in **Class room**, and shares the code; students join with the code and their roll number.
- Open **Session report** at the end and use **Print / save as PDF**.

7. Instructor guide

Learning objectives

- Identify bottlenecks and justify capacity with throughput economics.
- Set EOQ / ROP / safety-stock policies that balance holding, ordering and stockout costs.
- Compare dispatching rules on flow time, WIP and tardiness.
- Apply the cost-of-quality framework (prevention vs appraisal vs failure).
- Staff service systems using utilization, Erlang C, Little's Law and service targets.
- Design supply-chain resilience (buffers, dual sourcing, contingency).
- Use what-if analysis and optimization to evaluate operational decisions.

Suggested 75-minute session

Minutes	Activity
0–10	Set the role (students are operations managers). Show decisions vs scenario inputs.
10–55	3–4 topics, about 10 min each: review the concept, ask students to predict the best decision, run the defaults, let teams change one decision at a time, then open Get AI advice and discuss the trade-offs.
55–70	Stress test: change a scenario input (demand +30%, longer disruption). Is the recommendation still best?
70–75	Open the Session report, discuss the lessons, and print or save it for the record.

Discussion prompts

- Why does adding a Cutting machine change nothing? Where does the bottleneck move next?
- Why does the agent's ROP differ from the 95% textbook ROP?
- Why is the makespan identical for every sequence?
- At what external failure cost does 100% inspection pay off?
- Why does the 5th server cut waiting far more than the 6th?
- Set the disruption duration to 0. Which strategy wastes the most money?

8. Validation

An automated Playwright test drives the real page in Chromium and checks:

- Every topic: Run, Run again (identical results, so deterministic), KPI cards free of NaN, chart and diagnosis rendered.
- Every input in every topic (56 inputs, including the job table, rule selector, custom sequence and checkbox) changes the results or the agent’s recommendation. Inputs that matter only in some situations (e.g. backup premium when no backup is used) say so in their help text.
- Get AI advice, Hide advice, Apply recommendation → Run again reaches 100% of the best option in every topic.
- Input validation: an invalid custom sequence is rejected with a message; out-of-range numbers are clamped; an unstable queue is diagnosed.
- Reset topic, Reset all, Session report, Instructor guide and theme toggle work; no JavaScript errors; no horizontal page scroll at 390 px phone width.
- Live agent, tested against a local test server that imitates the Claude API: correct headers and body; all tool results returned in one message with matching ids; assistant content (including thinking blocks) passed back unchanged; the simulation tool returns engine numbers ($1,920); bad inputs return is_error; the sandbox leaves inputs untouched; Apply suggested settings works; a 401 error is shown and rolled back; chats are kept per topic; Remember/Forget key works; no network calls without a key.
- OpenAI-compatible path, tested end to end through the real local proxy against a fake NVIDIA-style server: request shape and Bearer auth; tool calls and tool messages with matching ids; JSON-string arguments parsed (invalid JSON reported to the model); reasoning hidden; automatic grounded mode for a model that rejects tools; NVIDIA preset defaults; a clear message when the browser call is blocked; Ollama works without a key; the proxy rejects hosts outside its allowlist and answers CORS preflight.
- Campaigns, tested on all seven topics: setup, locked scenario, hidden supply outcome, four rounds played with the official round scenarios, finite values, summary, mode switching, new campaign, yearly multipliers and the session report; the server accepts official rounds and rejects tampered ones.
- Scenario Agent and Ollama, tested against a fake Ollama server: one-click connection picks a chat model and skips embedding models; generated case applied with out-of-range values fitted, decision and unknown inputs dropped and listed; brief sent to the model; 14 kinds of broken JSON repaired (including missing commas between items and truncated output); JSON mode requested and dropped automatically when refused; self-correction with the previous reply; unusable output reported after two correction attempts; AI-written campaigns played round by round and flagged as practice; supply outcome hidden until played; the chat agent sees the current case; Ollama routed through the class server.
- Case data, tested on all seven topics: brief and exhibits without missing values, CSV download, live updates when inputs or job data change, demand history matching the planning mean and deviation but distinct from the simulated days, actual demand exhibit matching the simulation, observation log and time-study consistency, campaign rounds with hidden outcomes, generated case stories, and remembered collapse state.
- Time Series lab, tested end to end on all five datasets with correct decisions (7–8/8 each), a deliberately wrong path (steps 1–4 flagged, slow-decay warning, going back to Step 3), over-differencing detection, the agent refusing to advance without a decision, ADF answers locked until the test is run, mystery mode, new samples, CSV export and the session report.
- Study Room, tested with the five course decks and a fake Ollama: slide counts match python-pptx; images display; the course footer never appears in slide text; agent notes, glossary, code blocks, notes download, slide viewer, diagrams, quiz scoring and retry, and slide-grounded help; Ollama connection from the side panel; AI notes in parts, AI diagram (JSON mode), AI quiz validation (malformed question dropped, text answer mapped to index) and grounded AI help; PDF, Word, Excel, CSV, HTML and text files read; deck removal; practice links.
- Class room, tested end to end with a real server and three browsers: room creation with a class list; joining from a file:// page and from the served page; automatic run submission; practice runs

for changed scenarios; roll numbers outside the class list and duplicate devices refused; a forged result ignored (server re-computed 70%); dashboard, common mistakes, leaderboard order and CSV export; device reset; closed room; runs queued while the server was down and delivered after restart; state kept across page reloads.